

Activation Quantization

Ahmad Fraij

Issues with SINQ Quantized Model

- The last layer, the output projection layer (ff_out) was being quantized (left) because I forgot to ignore it in the pitch
- This is fixed by not quantizing ff_out layer as seen on the right

```
LLaDAModelLM(  
  (model): LLaDAModel(  
    (transformer): ModuleDict(  
      (wte): Embedding(126464, 4096)  
      (emb_drop): Dropout(p=0.0, inplace=False)  
      (ln_f): RMSLayerNorm()  
      (blocks): ModuleList(  
        (0-31): 32 x LLaDALlamaBlock(  
          (dropout): Dropout(p=0.0, inplace=False)  
          (act): SiLU()  
          (attn_out): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (ff_out): SINQLinear(in_features=12288, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (rotary_emb): RotaryEmbedding()  
          (attn_norm): RMSLayerNorm()  
          (ff_norm): RMSLayerNorm()  
          (q_proj): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (k_proj): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (v_proj): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (ff_proj): SINQLinear(in_features=12288, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (up_proj): SINQLinear(in_features=4096, out_features=12288, bias=False, device=cuda, W_q.device=cpu)  
        )  
      )  
      (ff_out): SINQLinear(in_features=4096, out_features=126464, bias=False, device=cuda, W_q.device=cpu)  
    )  
  )  
)
```

```
LLaDAModelLM(  
  (model): LLaDAModel(  
    (transformer): ModuleDict(  
      (wte): Embedding(126464, 4096)  
      (emb_drop): Dropout(p=0.0, inplace=False)  
      (ln_f): RMSLayerNorm()  
      (blocks): ModuleList(  
        (0-31): 32 x LLaDALlamaBlock(  
          (dropout): Dropout(p=0.0, inplace=False)  
          (act): SiLU()  
          (attn_out): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (ff_out): Linear(in_features=12288, out_features=4096, bias=False)  
          (rotary_emb): RotaryEmbedding()  
          (attn_norm): RMSLayerNorm()  
          (ff_norm): RMSLayerNorm()  
          (q_proj): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (k_proj): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (v_proj): SINQLinear(in_features=4096, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (ff_proj): SINQLinear(in_features=12288, out_features=4096, bias=False, device=cuda, W_q.device=cpu)  
          (up_proj): SINQLinear(in_features=4096, out_features=12288, bias=False, device=cuda, W_q.device=cpu)  
        )  
      )  
      (ff_out): Linear(in_features=4096, out_features=126464, bias=False)  
    )  
  )  
)
```

Effect of Unquantizing the ff_out

Flip % with quantized ff_out

correct/incorrect answers of the baseline model that changed.						
Model	ARC-Challenge	ARC-Easy	PIQA	Winogrande	MMLU	HellaSwag
LLaDA-8B-Base	2.99 / 1.45	2.27 / 1.52	1.80 / 0.87	3.95 / 3.24	6.52 / 6.65	1.20 / 0.45

Flip % with unquantized ff_out

answers of the baseline model that changed.						
Model	ARC-Challenge	ARC-Easy	PIQA	Winogrande	MMLU	HellaSwag
LLaDA-8B-Base (SINQ)	1.88 / 1.28	1.60 / 1.30	1.03 / 0.49	2.84 / 2.84	3.85 / 4.37	0.64 / 0.28

AWQ Quantization and Evaluation

- Quantized Llada and Dream models using AWQ and using 128 samples from the pile dataset with max sequence of 2048

4-bit quantization of Dream-7B-Base and LLaDA-8B-Base. The first/second number indicates the % of correct/incorrect answers of the baseline model that changed.

Model	ARC-Challenge	ARC-Easy	PIQA	Winogrande	MMLU	HellaSwag
LLaDA-8B-Base (SINQ)	1.88 / 1.28	1.60 / 1.30	1.03 / 0.49	2.84 / 2.84	3.85 / 4.37	0.64 / 0.28
LLaDA-8B-Base (AWQ)	2.90 / 0.85	1.60 / 1.64	0.82 / 0.76	3.24 / 2.84	4.56 / 4.24	
Dream-7B-Base (SINQ)	3.67 / 2.56	2.06 / 1.35	1.85 / 1.52	3.39 / 2.29	—	—
Dream-7B-Base (AWQ)	4.10 / 2.13	2.06 / 1.18	1.47 / 0.92	3.08 / 2.05	—	—

Weight Activation Quantization Techniques

These results from the paper below shows that Duquant was performing the best among these three weight activation quantization techniques

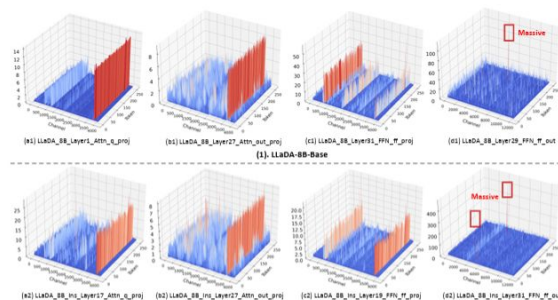
Table 2: Model performance on general tasks under weight-activation quantization.

Model	Setting	Method	WinoGrande	PIQA	ARC-C	ARC-E	MMLU 5-shot	Avg	Drop
LLaDA-8B	FP Model	-	69.9	74.6	46.4	71.1	65.7	65.5	-
	W8A8	SmoothQuant	67.7	70.8	45.5	70.5	65.0	63.9	↓ 2.5%
		QuaRot	68.6	71.1	45.2	70.8	66.1	64.4	↓ 1.8%
		DuQuant	67.9	70.4	45.9	71.4	66.0	64.3	↓ 1.9%
	W4A4	SmoothQuant	49.4	58.8	29.2	40.9	27.1	41.1	↓ 37.3%
		QuaRot	63.4	68.1	43.7	69.2	61.8	59.2	↓ 6.6%
		DuQuant	64.9	69.3	42.8	70.0	64.0	62.2	↓ 5.1%
LLaDA-8B -Instruct	FP Model	-	70.2	71.3	54.3	75.9	64.0	67.1	-
	W8A8	SmoothQuant	69.6	72.1	53.5	75.9	64.0	67.0	↓ 0.2%
		QuaRot	69.1	71.3	54.1	76.2	64.1	67.0	↓ 0.3%
		DuQuant	68.8	71.8	54.6	76.2	63.6	67.0	↓ 0.2%
	W4A4	SmoothQuant	52.3	65.1	34.0	54.0	32.8	47.7	↓ 29.0%
		QuaRot	65.2	69.8	51.3	75.1	61.1	64.5	↓ 3.9%
		DuQuant	66.4	72.2	52.7	74.8	61.2	65.4	↓ 2.5%

H. Lin *et al.*, “Quantization Meets dLLMs: A Systematic Study of Post-training Quantization for Diffusion LLMs,” Oct. 15, 2025, *arXiv*: arXiv:2508.14896. doi: [10.48550/arXiv.2508.14896](https://doi.org/10.48550/arXiv.2508.14896).

DuQuant

- Rotation-based quantization technique
- Uses outlier-aware rotation matrices and channel permutation to better align the activation distribution with quantization-friendly structures.
- These plots show the existence of activation outliers along layers of Llada model



[1]

DuQuant + SINQ

DuQuant first redistributes activation outliers before quantization then SINQ applies weight quantization (the activations were not quantized in this experiment, I just did it to see the effect of redistributing the activations and how it can deal with outliers)

- How it works
 1. Pass calibration data through the model
 2. Collect activations statistics
 3. Compute rotation matrix using DuQuant
 4. apply rotation matrix to weights
 5. Use SINQ on redistributed weights
 6. At inference time the input x is also rotated ($x @ R$) before the matmul

DuQuant + SINQ Results

Some results of DuSINQ approach (other benchmarks are still running)

4-bit quantization of Dream-7B-Base and LLaDA-8B-Base. The first/second number indicates the % of correct/incorrect answers of the baseline model that changed.

Model	ARC-Challenge	ARC-Easy	PIQA	Winogrande	MMLU	HellaSwag
LLaDA-8B-Base (SINQ)	1.88 / 1.28	1.60 / 1.30	1.03 / 0.49	2.84 / 2.84	3.85 / 4.37	0.64 / 0.28
LLaDA-8B-Base (AWQ)	2.90 / 0.85	1.60 / 1.64	0.82 / 0.76	3.24 / 2.84	4.56 / 4.24	
LLaDA-8B-Base (DuSINQ)	1.62/1.19		1.09/0.54	2.84/2.37		
Dream-7B-Base (SINQ)	3.67 / 2.56	2.06 / 1.35	1.85 / 1.52	3.39 / 2.29	—	—
Dream-7B-Base (AWQ)	4.10 / 2.13	2.06 / 1.18	1.47 / 0.92	3.08 / 2.05	—	—

Plan for next week

- Report perplexity results with all quantization techniques
- Fix issues with GPTQ and add it to the comparison with AWQ and SING
- Implement weight activation quantization (W4A8,W4A4). Start with SING and do simple activation quantization with it.
- Implement weight activation quantization on duquant and other techniques and compare the results